



Cairo University

Faculty of Computing and Artificial Intelligence

Department of Computer Science



جَامِعَةُ الْقَاهِرَةِ
CAIRO UNIVERSITY

Software Testing: Assignment 1 – Part 2

TA: Hassan Mourad

Implemented by

20221050	Daniel Sameh Kamal
20220388	Youssef Ehab
20220517	Michael Reda Moussa

Status Report: Privacy-First AI Chat Infrastructure

1. DigitScrubber (IScrubDigits)

Successful Tests (5 out of 6 tests passed):

The class masks generic digits, correctly handling single instances, multiple consecutive or non-consecutive instances, and accurately throws the expected **IllegalArgumentException** for blank strings and **NullPointerException** for null strings.

Test Method Name	Example
test_scrubDigits_successfully	"123-456" => "XXX-XXX"
test_scrubDigits_numbersOnlyInput	"1234567890" => "XXXXXXXXXX"
test_scrubDigits_multipleNumbersInput	"1234567890 and also 548482" => "XXXXXXXXXX and also XXXXXX"
test_scrubDigits_throwsException_whenInputIsEmpty	" " => IllegalArgumentException
test_scrubDigits_throwsException_whenInputIsNull	Null => "NullPointerException"

Unsuccessful Tests (1 out of 6 tests failed):

- test_scrubDigits_prices:

Justification: The test expects numerical prices to remain unmasked (for example 120\$ or \$120 should not become XXX\$ or \$XXX). However, the test failed because prices were masked. The root cause lies in **DigitScrubber.java**, which unconditionally replaces all numeric digits using **input.replaceAll("\\d", "X")** without implementing the logic that excludes numbers followed or preceded by a \$ sign.

2. EmailScrubber (IScrubEmails)

Successful Tests (4/6 tests passed)

The basic tests passed successfully. The class scrubs valid and standard email addresses correctly. It ignores non-email strings, and correctly throws a **NullPointerException** when a null input is passed.

Test Method Name	Example
test_scrubEmails_correctEmail	"user.name@gmail.com+first.last@sub.domain.co.uk" => "[EMAIL_HIDDEN] + [EMAIL_HIDDEN]"
test_scrubEmails_emailWithSpecial Characters	"dan+@mail.com" => "[EMAIL_HIDDEN]"
test_scrubEmails_invalidEmail	"notanemail + user@domain" => "notanemail + user@domain"
test_scrubEmails_throwsExceptionwhenInputIsNull	Null => NullPointerException

Unsuccessful Tests (2 out of 5 tests failed):

- test_scrubEmails_throwsException_whenInputsEmpty

Justification: The test should expect an **IllegalArgumentException** in the case of an empty string. However, it fails with a **NullPointerException**. Looking at **EmailScrubber.java** reveals the defect: the code uses a combined condition **if(input==null || input.isBlank())** and throws a **NullPointerException** for both cases, violating the explicit requirement to throw **IllegalArgumentException** for blank inputs.

- public void test_scrubEmails_emailWithNumbers()

Justification: The test should expect an **[Email_Hidden]** in the case of an email containing numbers. However, it fails by not scrubbing it. Looking at **EmailScrubber.java**, The regex **[a-zA-Z0-0._%+-]+@[a-zA-Z0-0.-]+\.[a-zA-Z]{2,6}** restricts numbers to **0-0** instead of a full **0-9** range. Any email containing digits from **1-9** (for example **player1@gmail.com**) will **NOT** be scrubbed, leading to a catastrophic /PII leak.

3. MainScrubber (IScrub)

Successful Tests (9/9 tests passed):

The behavioral tests of the requirements were extremely successful here. The tests accurately used Mocks to verify the method invocations (using **times(1)** and **never()**). For all three modes (**ONLY_DIGITS**, **ONLY_EMAILS**, **FULL_SCRUBBING**), the exact dependencies are used the correct number of times and in the proper sequential order.

Test Method Name	Example
testScrub_onlyDigitsMode_callsDigitScrubberOnceWithOriginalInput	Input "hello 123", ONLY_DIGITS; verifies digit mock is called once, email mock never.
testScrub_onlyEmailsMode_callsEmailScrubberOnceWithOriginalInput	Input "hello test@email.com", ONLY_EMAILS; verifies email mock is called once, digit mock never.
testScrub_fullScrubbingMode_callsBothInCorrectOrderWithChainedInput	Input "hello 123 test@email.com", FULL; verifies digit mock feeds sequentially into email mock.
testScrub_nullInput_scrubberThrowsException_returnsNull	Input null; digit mock throws NullPointerException internally, verifies method safely returns null.
testScrub_blankInput_scrubberThrowsException_returnsNull	Input " "; digit mock throws IllegalArgumentException internally, verifies method safely returns null
testScrub_onlyDigitsMode_digitScrubberThrowsException_returnsNullAndEmailNotCalled	Input "hello 123", ONLY_DIGITS; digit mock errors, verifies returning null & email mock skipped.
testScrub_onlyEmailsMode_emailScrubberThrowsException_returnsNullAndDigitNotCalled	Input "test@email.com", ONLY_EMAILS; email mock errors, verifies returning null & digit mock skipped.
testScrub_fullScrubbingMode_digitScrubberThrowsException_returnsNullAndEmailNotCalled	Input "hello 123 test@email.com", FULL; digit mock errors, verifies email mock is entirely skipped.
testScrub_fullScrubbingMode_emailScrubberThrowsException_returnsNullAndDigitCalledOnce	Input "hello 123 test@email.com", FULL; digit mock succeeds but email mock errors, verifies safe catch.